

Map the API, Skip the *Wrapper*

Trace one API down to its raw HTTP calls, write the bare function, break it on purpose, then wrap it in a UI — no library required.

NAME

DATE

ENDPOINTS FOUND

1 Map the API Before You Import Anything

- Pick a wrapper library already in your stack — **Twitter**, **Notion**, **Slack**, whatever.
- Open its docs or source. Find the **real HTTP calls** behind the one feature you actually use.
- Write down `method`, `path`, required headers — same reduction as Mastodon.py to two routes.

```
POST /api/v1/media      → upload, returns id
POST /api/v1/statuses   → post, needs media_ids[]
```

ENDPOINTS

2 Write the Bare-Metal Function

- Use `post_toot()` as template — one function, plain `requests`, **no wrapper import**.
- Bearer token in `Authorization` header, every call.
- If your API needs two steps — upload, then reference the returned *ID* in call two.

YOUR FUNCTION

3 Add the Failure Case On Purpose

- Generate a token with **partial scopes** — drop one permission on purpose.
- Run your function. Watch for *silent partial success*, not a crash.
- Add `raise_for_status()` or a `status_code` check so the real error JSON surfaces.

```
media_res.raise_for_status()
# 403 → {"error": "outside authorized scopes"}
```

FAILURE SEEN

4 Wrap It in Streamlit

- Turn Section 2's function into a form: `st.text_input(type='password')` for token, `st.text_area()` for content.
- Add `st.file_uploader()` if your API takes media.
- Run `streamlit run yourfile.py`. Success state must print a **live clickable link** pulled from the response — not a hardcoded checkmark.

APP RUNS?

5 Count What You Didn't Install

- List every method the full wrapper exposes — *timelines*, *streaming*, *following*, *DMs*.
- List the **two or three endpoints** your tool actually calls.
- One sentence: the dependency weight you skipped — lines of code, transitive packages — by going straight to `requests`.

WEIGHT AVOIDED